

AMBA APB Protocol (Advanced Peripheral Bus)

Prepared by AadhithyaSB

Agenda

- ▶ What is APB?
- ▶ Signal Description
- ▶ Master-Slave Configuration
- ▶ Transfer examples (read/write/ Write Followed by Read)
- ▶ Error Response
- ▶ Operating states & timing
- ▶ APB versions & features
- ▶ APB vs AHB/AXI
- ▶ Advantages & Disadvantages
- ▶ Reference

What is APB?

- ▶ AMBA (Advanced Microcontroller Bus Architecture) was introduced by ARM in 1996. The first AMBA buses were the Advanced System Bus (ASB) and the Advanced Peripheral Bus (APB).
- ▶ APB is a member of the AMBA protocol family.
- ▶ It offers a low-cost interface designed for minimal power consumption and reduced interface complexity.
- ▶ APB is suitable for connecting low-bandwidth peripherals that do not require the high performance of a pipelined bus interface.
- ▶ The protocol is unpipelined.
- ▶ All signal transitions are synchronized to the rising edge of the clock, simplifying the integration of APB peripherals into any design flow.

Signal Description

► Clock, Reset, and Address Signals:

PCLK: The rising edge of PCLK synchronizes all transfers on the APB.

PRESETn: Active-low APB reset signal, typically connected directly to the system bus reset.

PADDR: APB address bus, driven by the peripheral bus bridge, with a width of up to 32 bits.

► Control Signals:

PSELx: Generated by the APB bridge for each slave, indicating the device is selected and a data transfer is required.

PENABLE: Signifies the second and subsequent cycles of an APB transfer.

PWRITE: When HIGH, indicates a write access; when LOW, indicates a read access.

► Data and Response Signals:

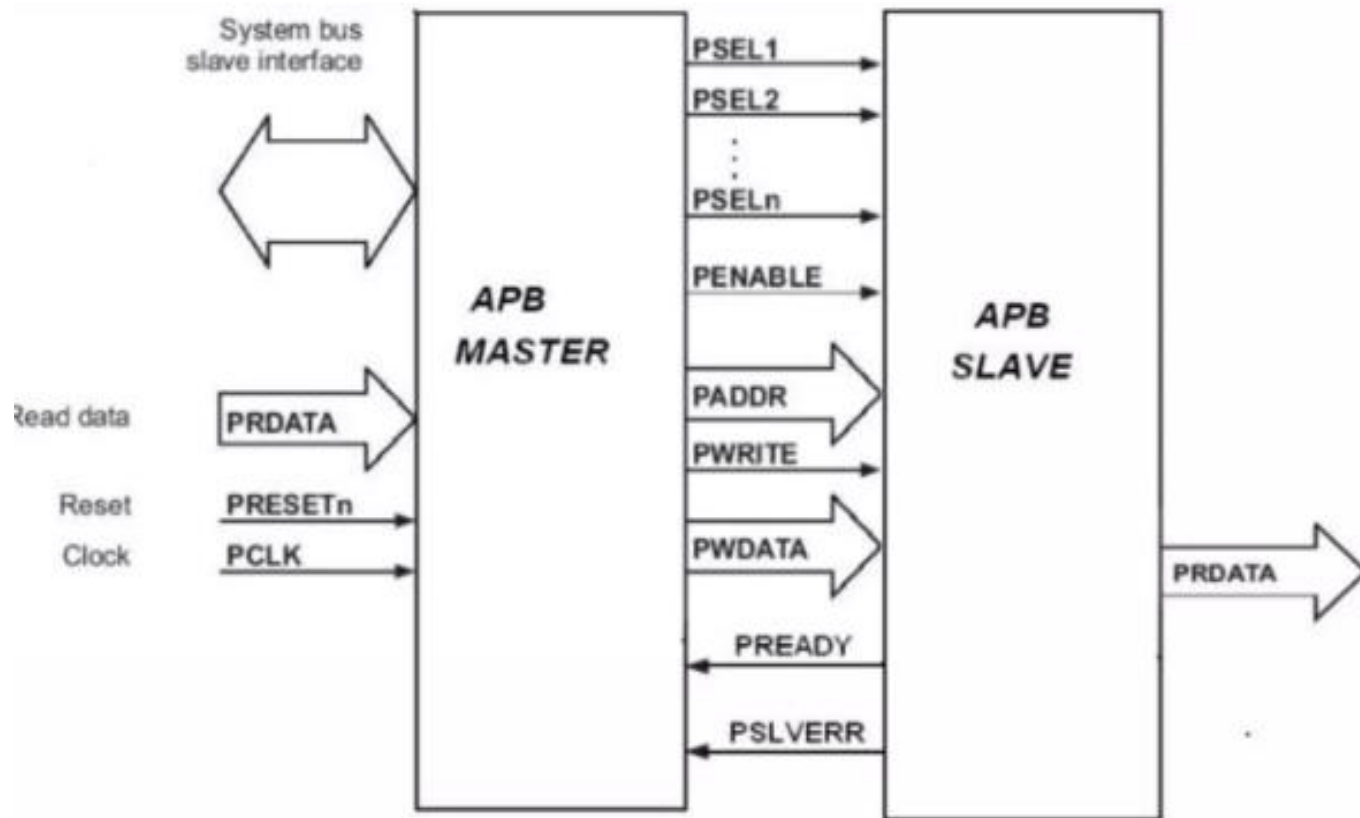
PWDATA: Driven by the bridge during write cycles (PWRITE HIGH), with a width of up to 32 bits.

PRDATA: Driven by the selected slave during read cycles (PWRITE LOW), with a width of up to 32 bits.

PREADY: Used by the slave to extend an APB transfer.

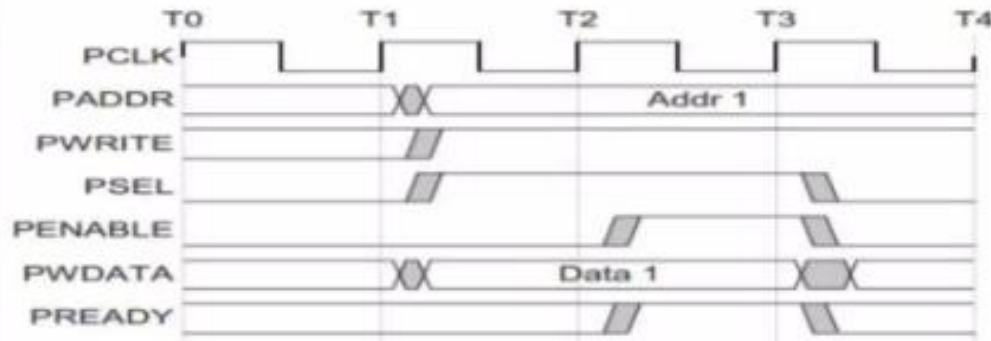
PSLVERR: Indicates a transfer failure. Note: APB peripherals are not required to support this pin. If unused, the bridge input should be tied LOW.

Master-Slave Configuration



Write Transfers

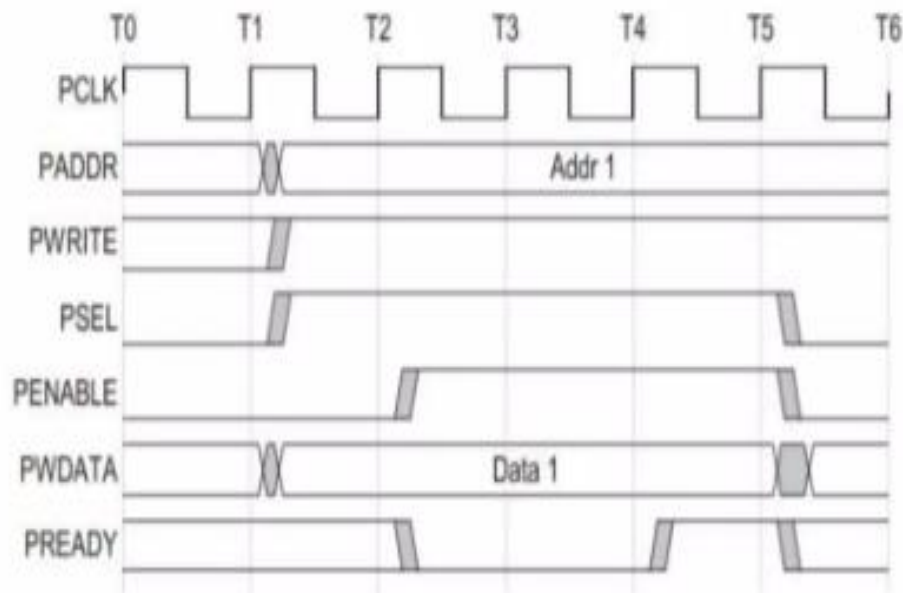
With no wait states:



- ▶ SETUP: PSELx=1, PWRITE=1, PADDR, PWDATA stable (PENABLE=0)
- ▶ ACCESS: PENABLE=1; slave samples PWDATA; PREADY=1; complete on this cycle

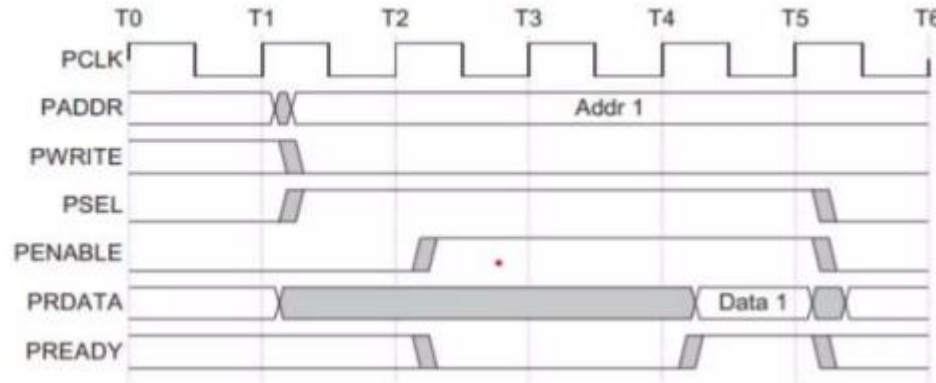
Write Transfers

► With wait state:



Read Transfers

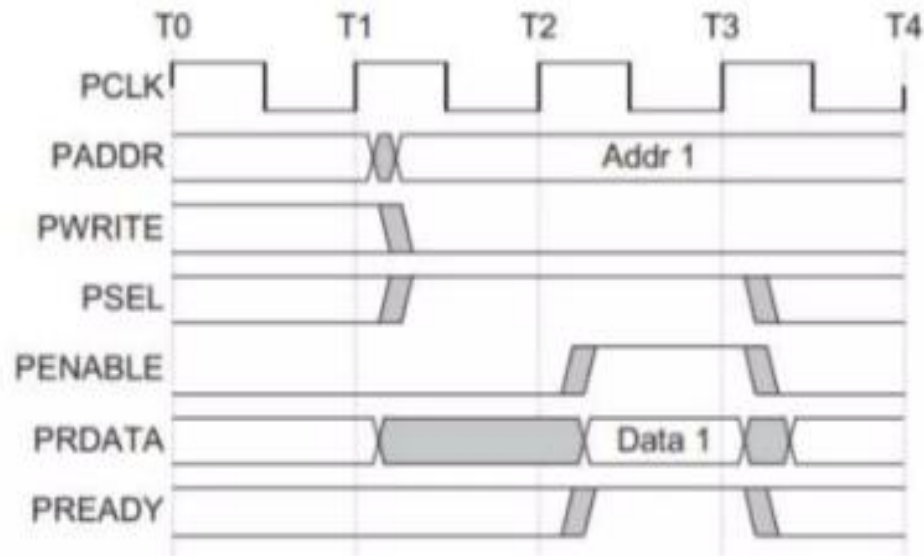
With wait state:



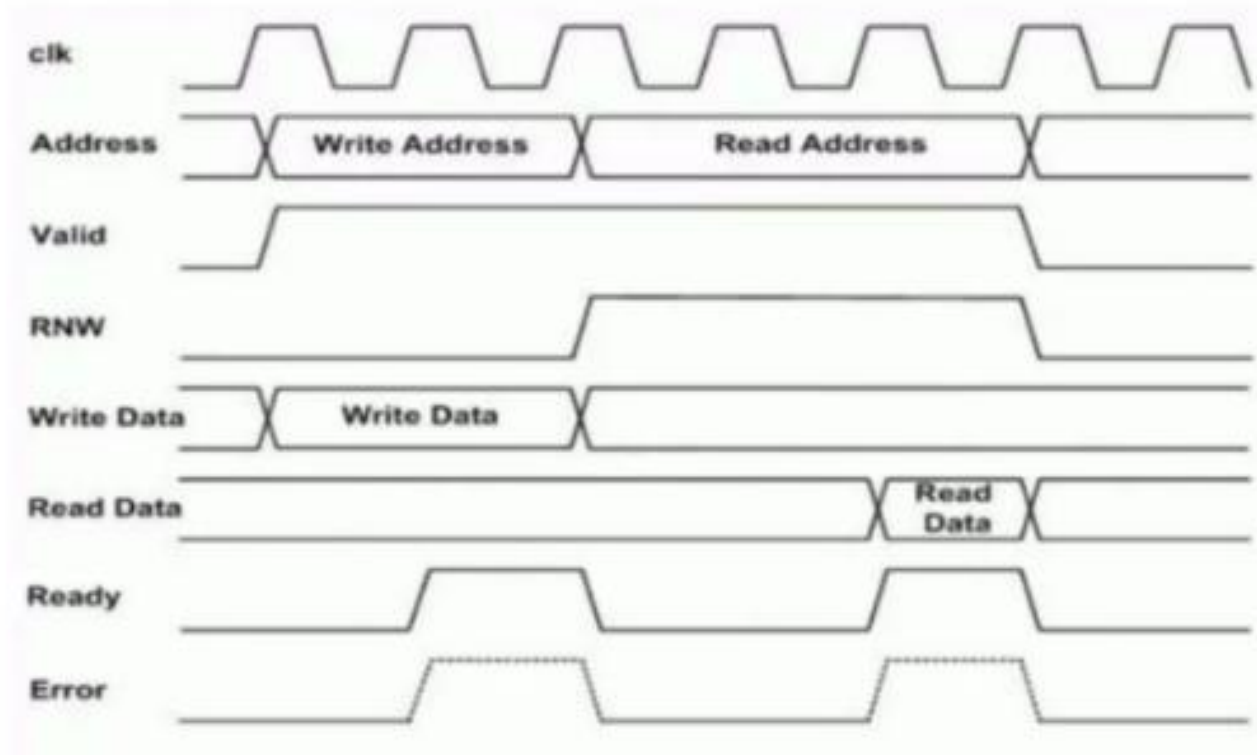
- ▶ 1) SETUP: PSELx=1, PWRITE=0, PADDR stable (PENABLE=0)
- ▶ 2) ACCESS: PENABLE=1; slave drives PRDATA; PREADY=1; master samples PRDATA

Read Transfers

With no wait state:



Write followed by read



Error response

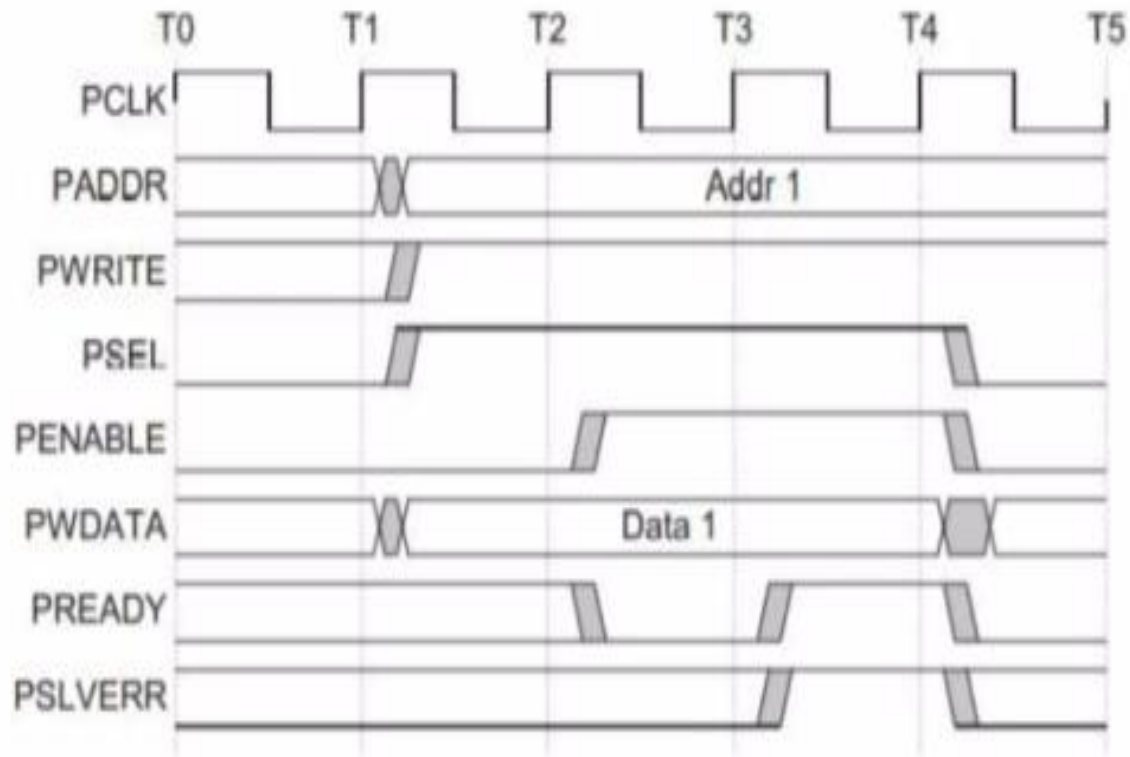
APB Transfer Error Handling (PSLVERR)

- ▶ **PSLVERR** indicates an error condition on both read and write APB transfers.
- ▶ The error signal is only valid during the **last cycle** of a transfer, when **PSEL**, **PENABLE**, and **PREADY** are all HIGH.
- ▶ It is recommended (but not mandatory) to drive **PSLVERR LOW** when not being sampled (i.e., when any of PSEL, PENABLE, or PREADY are LOW).

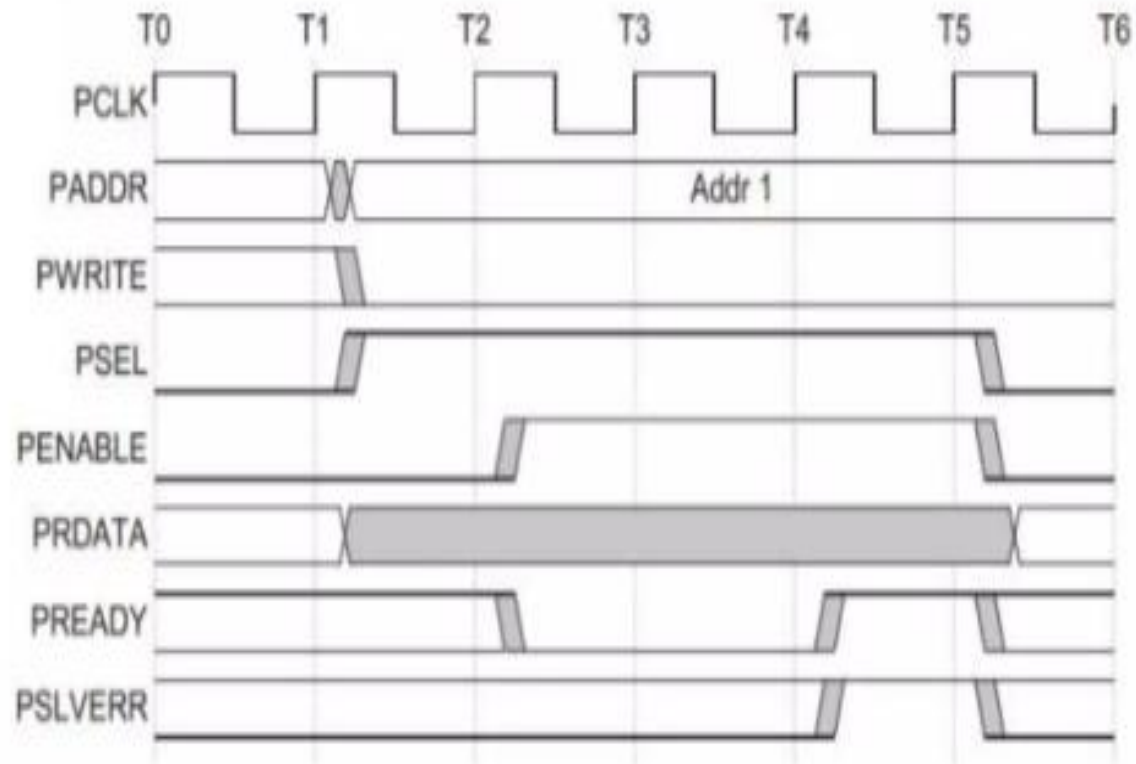
Error Implications:

- ▶ Transactions with errors **may or may not** change the peripheral's state (peripheral-specific).
- ▶ For **write transactions**, an error does **not guarantee** that the peripheral register was left unchanged.
- ▶ For **read transactions**, an error may return **invalid data** (no requirement to drive all 0s).

Failing Write Transfer



Failing Read Transfer



Operating States & Transfer Sequence

- ▶ APB transfer has three main states:
 - ▶ • IDLE : default state, no transfer
 - ▶ • SETUP : master asserts PSELx and provides address/control (1 cycle)
 - ▶ • ACCESS : PENABLE asserted; data transferred; slave drives PREADY/PSLVERR

Notes: PREADY low extends ACCESS (wait states); signals must remain stable while PREADY is low.

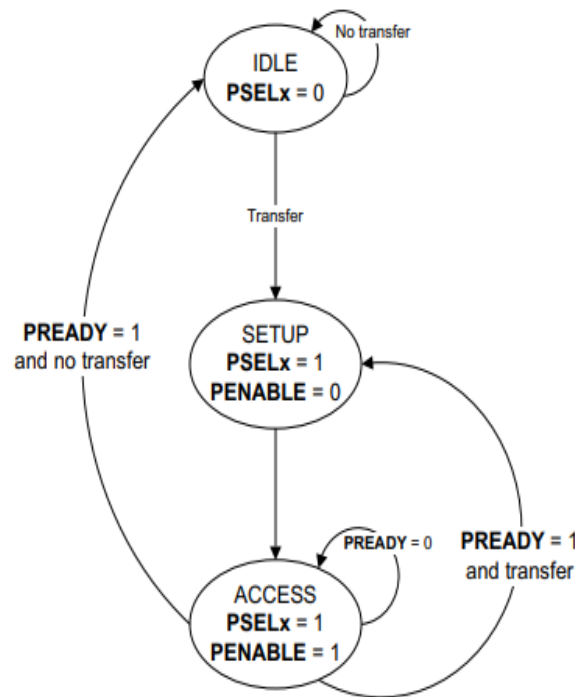
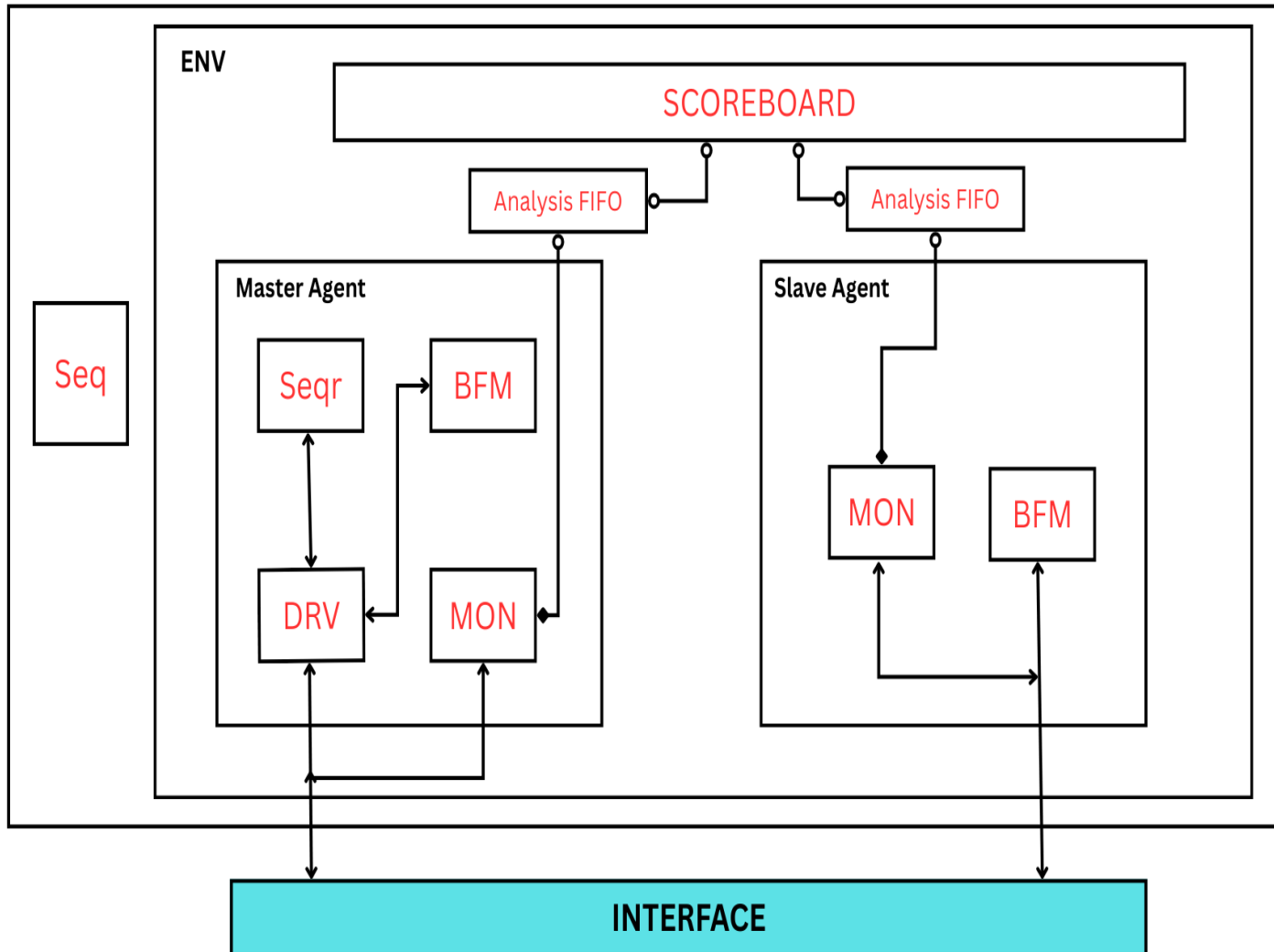


Figure 4-1 State diagram

APB Verification Plan



APB Versions & Notable Features

- ▶ APB2 / APB3 / APB4 / APB5 (AMBA evolution)
 - APB3/APB4 added features: protection (PPROT), write strobes (PSTRB), wake-up, parity, user signaling (varies by version)
 - APB4 is backward compatible with APB3; APB keeps interface simple for peripherals.

APB vs AHB / AXI (When to use)

- ▶ • APB: low-bandwidth, low-power peripherals (register access)
 - ▶ • AHB/AXI: higher performance, burst transfers, pipelined and/or out-of-order (use for memories, DMA, high-speed peripherals)
 - ▶ • Use APB behind an interconnect fabric (bridged from AHB/AXI) for control registers

Advantages

- ▶ Ideal for low-bandwidth peripherals: UART, I2C controllers, GPIO, timers, watchdogs.
- ▶ Low logic overhead, easy to implement, low-power.
- ▶ Slave-controlled wait states using PREADY for simple flow control.

Disadvantages

- ▶ **Low Bandwidth:** Not for high-speed data transfers.
- ▶ **No Burst Support:** Only single-word reads/writes.
- ▶ **High Latency:** Requires at least 2 clock cycles per transfer.
- ▶ **Single Master Only:** No multi-master support.

References

Key references used while preparing this presentation:

- ▶ ARM AMBA APB Protocol Specification (ARM Developer)
- ▶ Udemy: APB Protocol project